

# A protocol for verifying ecological analysis code developed with AI assistance

Masami Fujiwara, Department of Ecology and Conservation Biology, Texas A&M University

September 2026

## Contents

|                                                                     |    |
|---------------------------------------------------------------------|----|
| 0. Disclaimer . . . . .                                             | 1  |
| 1. Purpose and scope . . . . .                                      | 1  |
| 2. What goes wrong, and what finds it . . . . .                     | 2  |
| 3. Part A: a trustworthy run . . . . .                              | 5  |
| 4. Part B: the node census . . . . .                                | 10 |
| 5. Documentation . . . . .                                          | 15 |
| 6. Two practices that reduce what verification must catch . . . . . | 18 |
| 7. Limitations . . . . .                                            | 20 |
| 8. Summary of recommendations . . . . .                             | 23 |
| References . . . . .                                                | 24 |

## 0. Disclaimer

This document was drafted with assistance from Claude Fable 5 and revised with Claude Opus 5 between August and September 2026. The two R scripts that accompany it, `run_pipeline.R` and `compare_runs.R`, and the node enumeration prompt, `node_table_prompt.md`, were produced the same way.

The models drafted and revised text under my direction. Every claim, every recommendation, and every number in this document is mine, and I am responsible for all of them. The failure classes in Section 2 and the examples used throughout are observations from an analysis pipeline I wrote and verified. The protocol was developed in the course of that verification rather than proposed in advance of it.

## 1. Purpose and scope

Analysis code is increasingly written with the assistance of large language models, such as ChatGPT (OpenAI 2026) and Claude (Anthropic 2026). This raises a reporting problem: authors need to describe what they did to assure the code’s correctness, and reviewers need something they can assess. “The code was reviewed by the authors” is neither specific nor checkable.

This document sets out a protocol for verifying such code and for reporting that verification. It assumes a scientific analysis pipeline common in ecology, epidemiology, and the environmental

sciences: a series of scripts that read data, compute derived quantities, fit models, and produce the tables and figures a manuscript reports.

**Two questions: implementation and specification.** Verification asks (1) whether the code does what it is described as doing, and (2) whether that description is right. Call these implementation and specification. Most verification effort goes to the implementation question because many of its checks can be mechanized: code either runs or it does not, and every function, package, and file it names either exists or it does not. However, implementation issues are not all equally easy to detect. Some prove difficult precisely because the code runs without error. For example, code that computes a mean grouped by two variables when three were intended will complete successfully yet produce an incorrect result. Such implementation errors are as hard to spot as specification issues. Specification issues are generally harder to catch and more often alter conclusions. Both silent implementation errors and specification errors require careful checking, because a pipeline can execute cleanly, reproduce exactly, name nothing that does not exist, and still compute the wrong quantity from beginning to end.

The protocol presented in this document has two parts. Part A establishes that the reported numbers came from a documented run and do not depend on the session, the machine, or the moment they were produced; it is a precondition for both questions rather than an answer to either. Part B enumerates every transformation the pipeline performs and checks each one, asking of each both whether the code does what its description says and whether that description is right.

**Part B answers both questions.** Enumerating transformations, which Section 4 calls the node census, requires writing down for each one what it computes, at what grain, in what units, and with what settings. That record is also what the specification question needs. Judging whether a quantity was the right one to compute requires knowing precisely what was computed, and that information is normally scattered across scripts and memory. So the census does more than confirm that the code is correct: it makes the analysis’s own choices visible where they can be compared with one another.

**Not only for AI-assisted code.** The failure classes in Section 2 are ones we have met in code written without AI assistance, and they are not new: the same kinds of failure are documented in research code deposited well before language models were in use (Trisovic et al. 2022). What AI changes is their rate and their location, not their kind. So the checks that catch human error catch the model’s, and the protocol is, at its core, a protocol for verifying scientific analysis code of any origin. It is not specific to any language or model, although some problems particular to R (R Core Team 2026) are noted where they arise.

**What this protocol is not.** It is not a certification, and it does not establish that code is free of errors. It establishes that specific checks were performed and passed, and it makes those checks reproducible by a reader.

## 2. What goes wrong, and what finds it

The concern about AI-generated code is that it is confidently wrong in ways an author cannot easily see. The most cited form is invention: functions, arguments, and file names that do not exist. That form is real, and in our pipeline a required-input flag referred to a script that had never existed.

But it is also the form least able to survive execution. Part of this is a selection effect worth noting: code written interactively runs as it is written, so references to things that do not exist are caught at the console and rarely enter a codebase at all. The errors that cost us the most time were of other kinds.

What follows is a taxonomy of the failures that survive development, which is the population a verification protocol has to address. It is organized by how a failure is detected rather than by how it was produced. How an error entered the code is often not recoverable, and it never determines what finds it.

## 2.1 Implementation failures

We observed the classes below repeatedly in our own pipeline, except for the last (plausible substitution), which we treat as a hazard rather than an observation. From our experience, these errors lived in the connections between correct components rather than inside them. A model asked to write a function writes a good function, and asked for another writes another, but neither request establishes that the two agree about what they exchange. Code produced this way is locally excellent and globally inconsistent.

**(1) Duplicated constants that drift.** A constant defined in one script and retyped by hand in another, so that when the first changes, the second does not.

**(2) Scope assumptions.** Code that depends on something present in the calling environment, so it works when run one way and fails when run another. Two of our scripts located their output folder differently: one derived it from the working directory, the other from an absolute path set by an upstream script. Both were correct only when the pipeline was run by hand in the expected order.

**(3) Silent degradation.** Absence handled as a value rather than as an error. A join that matches nothing returns missing values, a filter that matches nothing returns no rows, and a conversion to numbers turns unrecognized entries into missing values. In each case, the run continues, and the empty result is reported as though it were a result like any other.

**(4) Inferred matching.** Code that guesses which column, file, or level was meant, usually by pattern-matching a name. When names change, it matches the wrong thing rather than nothing. Reading a directory and pairing its contents positionally with another directory is the same failure: the code takes whatever order it is handed as the order it needs.

**(5) Stale or invented references.** Paths, filenames, function names, arguments, and package versions pointing at what was renamed, removed, or never there. Whether a reference is stale or invented makes no difference to detection: both stop the run, provided the line containing them is actually reached.

**(6) Structure read as meaning.** How the data are arranged mistaken for what the data say. A quantity that describes a whole region, repeated on every row so that each subunit carries a copy, then analyzed as though each copy were a separate observation.

**(7) Vacuous validation.** A test that cannot fail. Ours checked an algebraic identity whose two

sides had been computed from each other, so they agreed by construction.

**(8) Plausible substitution.** A call that runs and returns something nothing downstream objects to, but is not the call that was intended. It takes two forms: a function or argument that exists and works but is not the one meant, and an argument omitted so that a real default silently takes its place. A mean grouped by two variables where three were intended is of the second kind. In both, the intended quantity was the right one and only the call was wrong.

Nearly all the implementation classes above are detectable in principle by something mechanical: run the code, resolve a name, compare two strings. Plausible substitution is the exception, and what finds it instead is the first of the questions asked at each node in Part B: whether the code does what its operation description says.

## 2.2 Specification failures

The classes below are different in kind. Each line is correct, and the quantity computed is not the one that should have been computed, so nothing in the run misbehaves and there is nothing for a machine to catch. This is where the errors that change what a paper concludes live.

The scope is narrower than that may suggest. Verifying an analysis does not mean verifying the methods it uses, and an established package's internals are out of scope, having been tested by their authors and their users. What is in scope is what the analysis passes into such a function and what it does with what comes back. That is where the analysis's own decisions live, and where a wrong number in a paper originates.

**(9) Wrong variant of a named method.** A real function called in a way that computes a relative of the intended quantity: a different bias correction, incidence data where the abundance form was meant, a target level the analysis does not specify. The package behaves correctly; the call asks it for something else.

**(10) Output handled wrongly.** The function returns what it should, and the code takes the wrong part, or the right part under a wrong assumption. A value pulled positionally from a table whose row order is not guaranteed, a log-scale quantity used where its exponential was meant, a column selected by a name that also matches something else.

**(11) Grain mismatch.** Quantities compared, divided, or joined that are not at the same spatial, temporal, or taxonomic scale. Two of our scripts decompose diversity at different grains, one from bay-pooled to region-pooled, the other from station mean to bay-pooled, so the upper level of the second sits where the lower level of the first does and the lower level of the second has no counterpart at all. Both called their levels alpha and gamma, setting up a comparison between a station mean and a bay assemblage and making a definitional gap look like a methodological difference.

**(12) Unmet assumption.** A method applied where its preconditions do not hold: an estimator extrapolating far beyond the observed sample, an interval assuming independence among nested observations, a trend fitted across a discontinuity in the sampling design.

**(13) Quantity substitution.** A reported number that is not the quantity the text describes: a median reported as a mean, a Shannon index on the log scale compared against Hill numbers, a

subset narrower than the Methods say.

**(14) Undistinguished absence.** A missing value that can arise for reasons which mean different things, where nothing downstream tells them apart. One of our functions returns an all-missing result under five conditions, among them an input with no rows and an estimator declining to extrapolate. The first says the group was never sampled; the second says it was sampled but too sparsely for the method to report a value. Both arrive downstream as the same NA. Where the conditions occur at different rates across groups or years, a trend can be driven by which one occurred where.

## 2.3 What follows for verification design

The detectors differ sharply in what they can see.

**Execution is strong for implementation and blind to specification.** Running from a clean session catches stale or invented references wherever the line containing them is reached, and catches scope assumptions because nothing is left over from a previous session. It does not catch plausible substitution, for the same reason it catches none of the specification classes: the code runs. Execution has nothing to say about whether the quantity is the right one.

**Reading is weak for implementation and primary for specification.** In a single file, duplicated constants, scope assumptions, stale references, and structure read as meaning are all invisible. Silent degradation and vacuous validation read as good practice. But the specification classes are errors of meaning, and reading is how meaning is checked.

**Reading in a fixed order, against a written record of what each step is supposed to produce, is stronger than reading a file.** That is the difference between Part B and ordinary code review, and Section 4 sets out how.

## 3. Part A: a trustworthy run

**Objective.** Establish that the numbers the manuscript reports depend only on the data and the code, and not on what happened to be in the R session when they were produced, on the machine that produced them, or on the day they were run.

**The steps.**

1. Set up a run so that nothing outside the code and the fixed inputs can reach it (Section 3.1).
2. Run the pipeline and record what it did (Section 3.2).
3. Before comparing anything, list the ways two runs of this code could differ (Section 3.3).
4. Run the pipeline twice more, at least one of those on a different machine, and compare the outputs (Section 3.4).
5. Where a quantity is expected to move between runs, measure how much and report it (Section 3.5).

Section 3.6 gives the reduced forms for pipelines that cannot support step 4 in full. Section 3.7 states what this part reaches that Part B cannot, and Section 3.8 lists what it produces and what it costs.

Two scripts accompanying this document, `run_pipeline.R` and `compare_runs.R`, carry out steps 1, 2 and 4 for a pipeline that is a set of R scripts run in a fixed order. Step 3 is not something a harness can do, but part of it can be delegated; Section 3.3 says which part.

### 3.1 Setting up a run

Run every script in order, each in its own R process started from the command line rather than typed into a console, from a session that loads no saved workspace, in a folder that contains only the code and the fixed inputs. The same practice is recommended to authors of published analysis code (Kellner et al. 2025).

The reason is that an R session accumulates objects. A script that appears to work may be reading a data frame someone created by hand an hour earlier, or an output folder path set by a script that is no longer in the pipeline. Neither is visible from reading the script, and neither survives a run that starts from nothing.

In practice this means copying the listed scripts and the listed data files into an empty folder and running there. Anything not listed is not copied, so a data file that was forgotten shows up as a file-not-found error rather than working quietly because it happened to be sitting in the project folder.

Keep out of the run any step that downloads data or queries a database. Such a step cannot produce the same output twice, which would make the comparison in Section 3.4 meaningless. Save its output once and treat that saved file as a fixed input.

### 3.2 Recording what the run did

Record five things.

**The environment.** The R version, the platform, and the version of every package the scripts use. A lockfile, meaning a single file listing every package and its exact version, is the convenient form, but a table of package names and versions serves.

**The output of each script,** both what it printed and what it wrote to the error stream, kept per script so a message can be traced to the step that produced it.

**Every warning and error,** collected into a table with a column you fill in yourself, marking each as expected (the code emits it deliberately), benign (a deprecation notice, a harmless partial match), or defect. Warnings are not noise. Four messages are defects by definition, because none can occur in code doing what its author intended: *could not find function*, *unused argument*, *object not found*, *no such file*. Each means the code named something that was not there.

**A list of every file the run produced,** with its size and a checksum. A checksum is a short string computed from a file's contents, such that two files with the same checksum are the same file. This is what makes the comparison in Section 3.4 cheap: most outputs can be compared without opening them.

**Confirmation that the outputs are the ones the manuscript reports.** A run that reproduces a different set of numbers than the manuscript is not a verification of the manuscript.

### 3.3 Auditing the sources of variation

If the code is fixed and the data are fixed, anything that differs between two runs came from outside both. That makes the list of possibilities short, and writing it down is the audit.

The possibilities fall into two families.

**Vary between runs on the same machine.** Random draws where no seed is set. Parallel workers that combine their results in whatever order they finish in. Any quantity or seed taken from the clock.

**Fixed on one machine, different between machines.** The locale, which is the regional setting that decides how text sorts, how string comparisons resolve, and how numbers are parsed from text. And the order in which files come back when code asks for the contents of a folder instead of naming the files it wants.

Summation order in floating-point arithmetic can arise either way, since it depends on how work was divided.

Do the audit in two passes, because the cheap pass is the one that must not be skipped.

**A mechanical scan.** Search the code for the mechanisms above. In R these announce themselves by name: `sample`, `runif`, `rnorm`, `rbinom` and their relatives, `set.seed`, `Sys.time`, `Sys.Date`, `list.files`, `dir`, `Sys.glob`, `sort`, `order`, and whatever package supplies parallelism. This is a text search across the codebase and takes minutes. It scales with the size of the code, not with the number of quantities the manuscript reports.

**A disposition for each hit.** Two questions: can this reach a number the manuscript reports, and if it can, is it controlled or is it measured. Most hits fail the first question, sitting in exploratory code or feeding nothing that is reported, and are dismissed in a line.

None of these mechanisms is a defect in itself. An unseeded draw is not an error. It means the exact figures cannot be regenerated, and the obligation is discharged by reporting the size of the variation instead (Section 3.5). A folder listing is not an error either. It becomes one when the order affects a result and nothing in the code fixes it.

Do the audit before comparing the runs, not after. It says in advance which quantities should match exactly, which should move, and roughly by how much, which is what makes the comparison readable. Without it, a sampler that reseeded itself and a file-ordering problem on the second machine present in exactly the same way: numbers that differ.

**Delegating the second pass.** The scan should not be delegated. A text search is exhaustive over the names it is given and a model is not, so a model asked to scan may quietly skip a file. What a model is useful for is the blind spot the scan leaves: a draw inside a helper the analysis wrote itself, a function passed as an argument and called through `do.call`, a package that parallelizes by option rather than by call. Finding those requires reading the code rather than searching it.

Frame the task accordingly. Give the model the code and the list of names the search covers, and ask what a search on those names could not have found. “Audit this pipeline” invites a plausible-looking list; “here is what a search on these names would match, what would it miss” is a question

with a checkable answer, and it is checkable without the search having been run first. The prompt we use is provided with this document as `variation_audit_prompt.md`.

Two cautions. First, whether a package draws or parallelizes internally is a fact about that package’s version, and a model’s knowledge of it is uneven and may predate the version in use. Treat anything it asserts about library behaviour as a lead to verify in the documentation, not as a finding. Second, an audit fails by omission, and a confident, complete-looking list makes a reader less likely to keep looking. The check on that is the same-environment verification run in Section 3.4, since two runs that disagree show a source was missed. That check is exactly what the one-run reduced form in Section 3.6 gives up, so delegating the audit and reducing to a single verification run should not be done together.

**What not to delegate: the disposition.** Whether a mechanism can reach a reported quantity is a question about the pipeline’s dataflow, and it is the same question Part B’s **inputs** column answers. Where the node table already exists, read the disposition off the table rather than asking anyone, model or person, to reconstruct it. This is a small dividend of the census: it settles the audit’s hardest question as a side effect.

### 3.4 Running twice more, and comparing

The numbers in the manuscript came from some execution, and that execution is the baseline. Verification is two further runs compared against it. This is two additional runs, not three. The manuscript’s own run counts, and a protocol that asks for a fresh set on top of it will be ignored by the projects whose pipelines take days.

**At least one of the two goes on a different machine.** The two comparisons answer different questions, and neither substitutes for the other.

A repeat on the same machine holds the environment fixed, so anything that differs came from inside the runs: leftover session state, order dependence, randomness, or logic that reads the clock. This repeat is also the completeness check on the audit, since two runs in one environment that disagree mean a live source was missed.

A run on a second machine is the only comparison that can reach the locale and the file ordering, because those are exactly what a same-machine repeat holds fixed. Such a repeat agrees with itself every time and says nothing about either. The cost is that the R version, the packages, the linear algebra library and the parallel backend all vary at once, so a difference does not name its own cause. Compare the environment records from the two machines first, since a package version difference explains a numeric one (Gronenschild et al. 2012).

**The agreement rule. The runs need not agree exactly. The difference between them must be small relative to the uncertainty the manuscript reports for that quantity.** A parameter reported as 0.42 with a standard error of 0.03 is the same result at 0.4213 and at 0.4219, and a different result at 0.48. A quantity reported with no uncertainty at all must match exactly, which is the strict half of the rule and covers sample sizes, counts, and every deterministic entry in a table.

This is a judgement applied to a written comparison, not a test with a cutoff. In practice the

comparison is rarely close: differences are either in the trailing decimals or they are plainly defects. Three situations arise.

- **A difference the audit predicted, of about the size it predicted.** A pass, recorded as one.
- **Two runs agree and the third does not.** The disagreement locates itself. Investigate the odd run rather than setting it aside; which machine was the odd one is itself the finding.
- **A difference that passes the rule but is larger than the audit led you to expect.** This is a finding rather than a pass. A parameter estimate that moves in the second decimal between two machines has told you something about how well the optimum is identified, and that belongs in the paper whether or not it clears a threshold.

### 3.5 Measuring stochastic variation

Do not treat identical output as the goal. A Monte Carlo estimate that moves in the fourth decimal between runs is a correct result reported at the wrong precision, not a defect.

Where the audit found a live random source reaching a reported quantity, measure the variation it produces and report it. Sometimes the pipeline supplies the measurement without extra work. In ours, a guild duplicated in the input was computed twice under different seeds, so the two results are independent estimates of the same quantity, and the difference between them gave the Monte Carlo standard error of every value in that table. Where nothing supplies it, repeat the stochastic step several times on one slice of the data.

Report the number. In our case one of the reported trends is smaller than the per-estimate noise, and a reader has no way to know that otherwise.

### 3.6 When the full procedure is not feasible

Long pipelines make the choice for you, and a requirement that cannot be met is a requirement that is ignored. Each reduced form below is legitimate, and each obliges the manuscript to say which was used.

**Two runs are too expensive: do one, and put it on the different machine.** The environment-dependent sources are the ones a reading of the code is least likely to have settled, and a same-machine repeat cannot reach them at all, so that is where a single run buys the most. What is given up is the completeness check on the audit. Say that one verification run was performed rather than two.

**The pipeline cannot be run again at all.** Rerun the stochastic step alone on one slice, which also produces the variation estimate in Section 3.5, and settle the deterministic remainder from the audit. The claim is weaker than any comparison of runs and should be stated as the weaker claim.

**The audit is too large to write out in full.** Keep the mechanical scan, which costs minutes and does not grow with the number of reported quantities. Restrict the written disposition to hits that reach a reported quantity, and trace any unexpected difference after the comparison rather than before it. What is given up is the ability to read a difference the moment it appears.

### 3.7 What Part A reaches that Part B does not

Part B checks a description of the pipeline. Part A checks that the pipeline described is the one that ran.

Part A also reaches code that produces no quantity and therefore has no node in Part B's table: error branches, refusal conditions, fallbacks. Such code fires on real data and changes what is reported by leaving something out. Nothing in a node table would show it.

### 3.8 Output and effort

**Output.** The environment record, the run logs, the table of warnings with their dispositions, the source-of-variation audit, and the comparison across runs with every difference dispositioned.

**Effort.** Under an hour of attention where the environment already works, plus whatever the pipeline takes to run, and roughly another hour to classify the warnings. The audit's scan takes minutes; its dispositions take under an hour for most pipelines, and neither grows with runtime. Environment setup is not counted, because it is a cost the project already carries; on a fresh machine it can exceed the whole of Part B while saying nothing about the code. In our case, almost all the elapsed time went to installing packages on machines that had never run the pipeline.

Although Part A is logically the precondition for Part B, a correction made late in the census invalidates these runs, so in practice they are best performed after the census is complete. Section 5.2 returns to this.

## 4. Part B: the node census

**Objective.** Enumerate every transformation the pipeline performs on its way to a reported number, and check each one.

This replaces the usual approach of selecting a few quantities to verify. Selection is a judgement about what matters, made before the checking that would inform it, and it misses whatever did not look important. A census does not select.

**The steps.**

1. Fix what counts as a node, and what region of the file each one occupies (Section 4.1, Section 4.2).
2. Enumerate the nodes into a table with a fixed set of columns, one script at a time (Section 4.3, Section 4.4).
3. Link the table into a graph and prune it back from the reported numbers (Section 4.4).
4. Check the nodes in forward order, asking four questions at each (Section 4.5).

Section 4.6 describes what assembling the table makes visible that checking the code does not, Section 4.7 states what the census does not cover, and Section 4.8 gives the granularity rule and the effort.

The table also produces, as it is built, a description of what the pipeline computes at a uniform level of detail. Where the Methods are not yet written, that description is most of their content.

## 4.1 What a node is

A node is a point where a quantity changes meaning, not where a variable changes value. A rename is not a node. Neither is a reshape. These are:

- an aggregation, join, or filter that changes what one row represents
- a call into a statistical package
- arithmetic combining two derived quantities
- a filter that changes which population is described

Three rules settle the cases where the same code appears more than once.

**Two steps performing the same operation, on the same inputs, with the same settings are one node.** The duplication is itself a finding, because nothing keeps two copies in agreement and a correction made at the node reaches only one of them.

**The same code applied to different inputs is two nodes,** because the inputs are what is being checked.

**A function definition called from several places is one node, and each call site is an additional node.** In our pipeline one coverage-standardization function is called on three different assemblages. The function is one node, its three call sites are three more, and the difference between them is the substance of the analysis.

## 4.2 The region a node occupies

Every node occupies a contiguous region of the file, delimited by a first and a last line copied word for word from the source. The region, not the single line, is what a reviewer reads when the node comes up for checking.

A region will usually contain statements that are not themselves nodes: a reshape, a rename, an intermediate object that carries a result to the next line. These are absorbed into the node whose decision they serve, and the region is what makes the absorption visible. Recording in one clause why a statement was absorbed keeps absorption from becoming a way of leaving code unread.

Regions must not overlap, and rows are listed in source order. A statement that falls outside every region is a claim that it belongs to no node, and that claim should be one you intend to make.

**Anchors, not line numbers.** A region is located by quoting its first and last lines, because line numbers drift with every edit and a line of code does not. The rules that make an anchor survive editing and be findable by a script:

- Copy the line from the source exactly, including its indentation and any trailing comma or pipe. Do not retype it, do not tidy the spacing, and do not rewrite `%>%` as `|>` or add or remove a `dplyr::` prefix.
- An anchor carries no line numbers, no file paths, and no comment text. A line whose only distinguishing feature is its comment cannot serve.
- Each anchor is unique within its file, or else is paired with a context line above it that makes the pair unique. Short generic lines are rarely unique on their own, and the assignment that

opens the block is usually the context line to reach for.

- Prefer lines a reformatter is unlikely to move. An assignment with a named function call is stable; a long line that a formatter might wrap is not.
- Avoid lines containing double quotes where possible, since a CSV has to escape them and a hand-edited table is easy to break.

The last line of a region is often a closing line such as `)`, which is almost never unique. That is expected. Keep the true region and supply a context line rather than shortening the region to obtain a tidier ending.

### 4.3 The node table

One row per node, in a plain-text table kept under version control alongside the code. The columns:

- **id**: a stable name, `script.quantity`. Other rows refer to it, so it must not change once cited.
- **script**: the filename.
- **section**: the existing section header in the script that the node falls under.
- **start\_anchor, start\_context, end\_anchor, end\_context**: the region, as defined in Section 4.2. The context columns are filled only where the anchor alone is not unique.
- **inputs**: the node ids this node consumes, or the source file for a node reading a fixed input. This is what makes the table a graph.
- **grain\_in, grain\_out**: what one row means on each side, for example `gear × bay × season × year × species`. Where these differ, the difference is the thing to check; where they should differ and do not, something is being double-counted.
- **units\_in, units\_out**: counts, individuals, Hill numbers, nats, proportions. This is where scale errors surface.
- **operation**: one sentence on what the node does and why.
- **settings**: the decisions embedded in it, including thresholds, target levels, bounds, tie-breaks, seeds, and any argument that changes what is computed. This column is the one that most directly becomes Methods text.
- **na\_behavior**: what happens when the operation cannot produce a value, and whether the causes are distinguishable downstream. Required for every node. Where a function returns a missing value for several different reasons that arrive downstream as the same **NA**, say so here explicitly, because undistinguished absence is invisible otherwise.
- **checked, verdict, notes**: the date and method, one of *ok*, *finding* or *question*, and the text of any finding. Left empty during enumeration.

### 4.4 Building the table

**Pass one: enumerate, one script at a time, in source order.** Read the script, mark the regions, and fill every column except the last three. Three rules govern what goes in:

- Describe what the code does, not what it should do. Where the code and its comments disagree, record the disagreement in **operation** rather than choosing between them.
- Where a region absorbs a statement that is not itself a node, say in one clause why.

- Where the meaning of a quantity cannot be determined from the code alone, write the question in **operation** rather than guessing. An unanswered question in the table is worth more than a plausible answer.

**Pass two: link, then prune backwards.** The **inputs** column turns the rows into a graph. Start from each reported number and walk back through its inputs until you reach a fixed input file. This bounds the census: everything a reported number depends on is enumerated, and anything the walk does not reach is a candidate for deletion rather than for checking.

Enumerating forwards through a file and pruning backwards from the results is deliberate. Reading a script in its own order is what a person can actually do without holding the whole pipeline in mind, and the backward walk is what keeps the census bounded by the manuscript rather than by the codebase.

**Then check the anchors mechanically, not by eye.** A short script confirms that every start and end anchor occurs in its file, that each is unique or has a context line, that the end anchor occurs at or after the start anchor, and that no two regions overlap. Report any anchor that could not be made unique rather than leaving it to be discovered later.

**Delegation and its limit.** Pass one is description, it is largely mechanical, and it can be delegated, including to a language model given the column definitions and the rules above. The prompt we use is provided with this document as `node_table_prompt.md`; it states the rules of Section 4.1 to Section 4.3 in the form a model needs, fixes the output format, and requires the model to report any anchor it could not make unique rather than adjusting a region to make the check pass.

Pass two and the checking in Section 4.5 are judgement and cannot be delegated. The reason is the same one that governs hand computation below: if a single source produces both the code and the account it is checked against, any misunderstanding enters both sides and the check passes. So a table produced this way is read against the code row by row before any node is checked. A description that is wrong in the same way the code is wrong is worse than no description, because it makes the error look confirmed.

## 4.5 Checking the table: forwards

Order the completed table so that every node appears after its inputs, and check in that order. This matters more than it sounds.

Checked backwards, every node raises questions that are answered upstream, and the reviewer holds them open while descending. Checked forwards, each node's inputs are already understood when it is reached, and the only new thing is the transformation itself. In our experience the same nodes take substantially longer backwards than forwards, and the forward pass also matches how errors propagate: verify the source before the transformation before the consumer.

At each node, four questions:

- Does the code in the region do what **operation** says?
- Is **grain\_out** right? Read what the operation groups or joins by, and confirm that one output row means what the column claims.

- Are the **settings** the intended ones? A setting that cannot be justified once written down is a finding.
- Does **na\_behavior** describe what actually happens, and is it acceptable?

**Where the operation is arithmetic that can be evaluated on paper, do so.** Choose cases where the computation reduces to something checkable: two assemblages sharing no species drive a multiplicative partition to its ceiling; a rarefaction target at or above the observed total makes the draw a no-op and reduces the computation to the underlying index on a known incidence matrix. Write the expected value down before running anything.

**The expected value must come from outside the code being tested.** If the same source produces both the code and the expected answer, any misunderstanding enters both sides and the test passes. Where AI assistance was used to write the analysis, it should not also be used to generate the values the analysis is checked against.

#### 4.6 What the census makes visible

A node table is a description of the analysis in one place, at a uniform level of detail, with every quantity's grain, units, and settings recorded beside every other's. Assembling it has an effect that checking the code does not: choices that were made separately, months apart, in different scripts, end up on adjacent rows where they can be compared.

Two examples from our pipeline, both raised by the census and neither a coding error.

**A divisor that differs between two levels of the same analysis.** The multiplicative partition divides an assemblage diversity by the number of assemblages. At community level the divisor is the number of bays *sampled*; at guild level it is the number of bays where the guild was *observed*. Each is defensible in isolation. The census put them on two rows with the same operation text and different settings, which is what made the difference a question rather than an accident of two authors on two days.

**A rescaling whose denominator is itself a result.** Guild beta is rescaled by its ceiling so that guilds occupying different numbers of bays are comparable. The ceiling is observed occupancy, which is itself changing over the study period, so a guild that contracts its range alters its own denominator. Recording that in one field, next to the note that the rescaled value is undefined for a guild in a single bay, is what turned a line of arithmetic into a methodological question about what the trend measures.

Neither question is answered by a verdict in the table. What the census does is produce them, in a form specific enough to argue about, and record the choice actually made so that a reader can disagree with it knowingly.

#### 4.7 What the census does not cover

A node census subsumes most of what would otherwise be separate activities. Reading the computational core is what checking a node is, organized by quantity rather than by file. Shared conventions become node attributes, so a category level used inconsistently appears as two nodes

disagreeing about the same field, which is stronger than a text search. Grain mismatch becomes structural rather than something a reviewer must remember to look for.

Two things it does not reach. Both are cheap, and both should be kept alongside it.

**A text search of the whole codebase.** The census is organized by quantity, and each quantity appears once. If the same wrong value was typed into four scripts, one node describes it and the table shows one instance, so correcting that node corrects one copy and leaves three the table cannot reveal. Searching the code as text finds all four. An hour spent this way is worth keeping.

**Code that computes nothing.** A node is a point where a quantity changes meaning, so an error branch, a refusal condition or a fallback has no node and appears nowhere in the table. Such code still changes what is reported, by leaving something out when it fires on real data. Only running the pipeline shows this, which is Part A's job.

## 4.8 Granularity and effort

**The granularity rule.** A script of about 800 lines usually yields 10 to 20 nodes. Many more than that means the granularity is too fine: merge neighbouring rows and let the regions grow. Many fewer means regions are absorbing decisions that should be visible as settings of their own. The count is a check on the enumeration, not a target.

The node count is always lower than an operation count would suggest, because the rules in Section 4.1 collapse repetition. In our pipeline one serial-correlation selection routine is used by six scripts and is one node with six call sites, not thirty separate rows.

**Effort.** Enumeration is largely mechanical. Checking, done forwards by someone who knows the data, runs at roughly a minute or two a node once the table is in front of you, so a few sittings cover a table of a hundred rows. Backwards it is several times that, which is the argument for building in one direction and checking in the other.

**A defensible partial application** is to enumerate everything and check exhaustively only the chains that reach reported quantities, marking the remainder as enumerated but unchecked. That is still a stronger statement than any sample, and it makes the unchecked portion explicit rather than invisible.

## 5. Documentation

**Objective.** Leave a record that lets a reader see which checks were performed and repeat them, and state in the manuscript what was done in terms specific enough to be questioned.

Section 5.1 lists what to keep and where. Section 5.2 covers when a verdict stops being valid and when the verification loop ends. Section 5.3 gives a template for the manuscript, and Section 5.4 covers what to say where a part was performed in reduced form.

### 5.1 The record

The node table is the audit record. Alongside it, keep the outputs of Part A listed in Section 3.8: the environment record, the run logs, the table of warnings with their dispositions, the source-of-

variation audit, and the comparison across runs.

Two further items belong with the record where they were used. If the enumeration in Section 4.4 or the reading pass of the audit in Section 3.3 was drafted with a language model, keep the prompt, because a reader cannot judge a delegated description without seeing what the model was asked for. If the runs in Part A were produced by a harness rather than by hand, keep the harness, because it is what determines which files were treated as fixed inputs and which messages were collected as warnings.

Keep all of it in version control beside the code, so that the verification and the thing verified move together, and complete it during the work rather than afterwards. A table filled in at the end records what someone remembers rather than what they did.

**Do not annotate the analysis scripts with review marks.** The scripts are a deliverable. Review annotations complicate the version history and confuse later readers, and they go stale silently. The node table’s anchors point into the code without changing it, which is the reason for the anchor convention in Section 4.2.

## 5.2 When a verdict expires, and when the loop ends

**A verdict applies to the code as it was when the verdict was given.** The protocol assumes a script is settled before its nodes are checked. Where a script is revised afterwards, clear the verdicts of its nodes and recheck them. The script column makes that a one-line filter, and the anchor check in Section 4.4 should be run again at the same time, since an edit that moves or reformats a line can break an anchor without changing what the node computes.

**Verification iterates, and it terminates.** A finding is corrected. The manuscript is amended where the correction changes a reported number. The nodes of the revised script are rechecked. Part A’s runs are performed again against the new baseline, because the numbers a manuscript reports must be the ones its last run produced. The loop ends when no finding remains uncorrected.

**Questions do not hold the loop open.** A question records a choice that survives into the published code. It is answered in the Methods and Discussion, not by a revision to a script, and an open question is reported rather than resolved away.

**On the order of the two parts.** Part A is logically the precondition for Part B, and the two are written here in the order they are read. They are not cheapest in that order. A correction made late in the census invalidates Part A’s runs, so in practice the verification runs are performed after the census is complete.

## 5.3 Reporting in the manuscript

Report specifics, not adjectives. A narrow claim that withstands questioning is worth more than a broad one that does not.

A template follows, in the order of the two parts. Square brackets mark what an author supplies; the supplementary materials are referred to by name and should be numbered according to the journal’s convention.

Analysis code was developed interactively with large language models ([models and roles]) under author supervision. Quality assurance had two parts.

First, the reported results come from a documented run ([R version; lockfile or package versions]) with [N] warnings, each accounted for. The pipeline was run [twice / once] more for verification, [one of these runs / that run] on a machine other than the one that produced the reported results, and the outputs were compared: [every reported quantity agreed exactly / differences were confined to [quantities], each small relative to the uncertainty reported for it, the largest being [size]]. The sources of run-to-run and between-machine variation were audited [in full / by a scan of the code for the mechanisms named in the protocol, with a written disposition for each that could reach a reported quantity]. [The reading pass of the audit was drafted with [model] from the search results and the code, and its findings were verified against package documentation.] The audit, the run logs, the environment record, and the scripts used to produce and compare the runs are provided as supplementary material.

Second, every transformation on the path from the input data to a reported quantity was enumerated as a node, giving [N] nodes across [N] scripts, each recorded with the region of code it occupies, its inputs, its grain and units before and after, its settings, and its behaviour on missing values. [The enumeration was drafted with [model] using the prompt provided as supplementary material, and every row was read against the code by the authors before any node was checked.] All [N] nodes were checked in forward order, of which [N] were verified against hand computation with expected values derived independently of the code. The node table, including [N] open questions on choices the census brought to light, is provided as supplementary material.

**Why the template reports coverage and open questions rather than findings.** A count of findings is worth little as evidence of thoroughness. A finding that has been corrected leaves no trace in the final table, so a thorough census and a careless one both report zero. What the record does show is how much was checked and what remains open, and those are the numbers a reader can do something with.

**The bracketed sentence on delegation is not optional where it applies.** A paper whose subject is the verification of AI-assisted code should say whether a model also wrote the description the code was verified against, and what was done about it. Leaving it out invites exactly the question the protocol exists to answer.

## 5.4 Reporting a reduced form

Each reduced form in Sections 3.6 and 4.8 is legitimate, and each obliges the manuscript to say which was used and why. The cases and the wording:

- **One verification run rather than two.** Say that one was performed and that it was on a different machine. What is given up is the completeness check on the audit, since two runs in one environment that disagree are what show a live source was missed.
- **The stochastic step repeated on a slice rather than the whole pipeline rerun.** Say that the deterministic remainder was settled from the audit. This claim is weaker than any

comparison of runs and should be stated as the weaker claim.

- **A scan rather than a full written audit.** Say that the disposition was restricted to mechanisms reaching a reported quantity, and that differences were traced after the comparison rather than predicted before it.
- **Nodes enumerated but not checked.** Give both counts and say which chains were checked exhaustively. This is still a stronger statement than any sample, and it makes the unchecked portion explicit rather than invisible.

A reduced form reported plainly is worth more than a full form claimed loosely.

## 6. Two practices that reduce what verification must catch

Everything above is applied after the code is written. Two habits applied while writing it remove whole classes of error before they can occur, and both cost less than finding the same errors later. Both belong to a wider body of recommended practice for scientific computing (Wilson et al. 2014).

The second of the two rests on a device that may be unfamiliar. An **assertion** is a line of code that states something which must be true at that point and stops the run if it is not. In R the usual form is `stopifnot()`:

```
stopifnot(nrow(catch) > 0)
```

If `catch` has no rows, the script halts and names the failed condition. Nothing else happens, nothing downstream runs, and no empty table is quietly written to disk. An assertion is not a comment and not a diagnostic message. Its whole value is that it refuses to continue.

### 6.1 Define each shared convention once

A convention is anything the analysis has decided and more than one script needs to know: the set of bays and the order they appear in, the gear codes that count as a valid sample, the years the study covers, the minimum sample size for inclusion, the random seed, the path to the data folder.

Put every one of them in a single file that each script reads at the top:

```
# conventions.R
BAYS      <- c("Sabine", "Galveston", "Matagorda", "San Antonio",
              "Aransas", "Corpus Christi", "Upper Laguna Madre")
YEARS     <- 1982:2019
MIN_TOWS  <- 5
SEED      <- 20260401
```

and at the top of every analysis script:

```
source("conventions.R")
```

The alternative, which is what most pipelines do by default, is to type the seven bay names into the four scripts that need them. That works until one of them changes. Then three scripts agree and one does not, nothing errors, and two tables in the manuscript describe slightly different study

areas. This is class (1) of Section 2, and defining the convention once removes it by construction rather than leaving it to be caught.

The habit pays a second time in Part B. A convention defined once is one node; a convention retyped in four places produces four rows that the census will show disagreeing about the same field.

## 6.2 Prefer stopping to degrading

The failure that costs the most is not the one that crashes. It is the one that continues. A join that matches nothing returns missing values, a filter that matches nothing returns an empty table, and `as.numeric()` turns an unrecognized entry into NA. In each case the run completes and the empty result is reported as though it were a result like any other. That is class (3) of Section 2, and the remedy is to write the check that refuses.

**Absence should stop the run, not produce a default.**

```
stopifnot(file.exists(path))
```

A missing input file that silently becomes an empty data frame will travel a long way before anyone notices.

**A lookup that matches nothing should stop, not return the nearest thing.** Code that guesses which column or which species code was meant, usually by matching part of a name, matches the wrong thing rather than nothing when names change. Name what you want and assert that it is there:

```
stopifnot("biomass_g" %in% names(catch))
```

**A category level outside the expected set should stop, not become missing.** A bay name misspelled in a new data delivery should halt the run and say so:

```
stopifnot(all(catch$bay %in% BAYS))
```

**A join should state the number of rows it expects.** A join that silently duplicates rows is one of the easiest ways to inflate a sample size without noticing:

```
joined <- merge(catch, traits, by = "species_code")
stopifnot(nrow(joined) == nrow(catch))
```

**A script should state what its inputs must look like.** The columns a data frame has to contain, and the types those columns must have, are what a script assumes of whatever produced them. Writing those assumptions down as a check means that a change upstream halts the run instead of propagating:

```
required <- c("bay", "year", "species_code", "n")
stopifnot(all(required %in% names(station_summary)),
           is.numeric(station_summary$n),
           is.character(station_summary$species_code))
```

This one earned its place during our own work. A script refused to run against output from an earlier version of the script before it and named the missing columns. Nothing had to be debugged, because nothing had run.

**A quantity with known bounds should be checked against them.** Many ecological quantities have limits that follow from their definition: a proportion lies in  $[0, 1]$ , a Shannon index of  $S$  species cannot exceed  $\log(S)$ , a multiplicative beta cannot exceed the number of assemblages. Check them.

```
stopifnot(all(beta >= 1 & beta <= n_bays, na.rm = TRUE))
```

**A folder read for its contents should have its file order fixed,** or the code should state that the order does not matter. `list.files()` returns files in an order that depends on the machine, and pairing two such listings positionally is a way of taking whatever order you are handed as the order you need. This is also one of the between-machine sources of variation in Section 3.3, so fixing it here removes an entry from the audit:

```
files <- sort(list.files(data_dir, pattern = "\\*.csv$"))
```

**Where a function can return a missing value for more than one reason, it should say which.** One of our functions returns an all-missing result under five conditions, among them a group with no rows and an estimator declining to extrapolate. The first means the group was never sampled; the second means it was sampled but too sparsely for the method to report a value. Both arrive downstream as the same NA, and where the two conditions occur at different rates across bays or years, a trend can be driven by which one happened where. That is class (14). Return the reason alongside the value:

```
list(value = NA_real_, reason = "coverage below target")
```

**An algebraic identity should be checked on quantities computed independently.** Verifying that  $\alpha$  times  $\beta$  equals  $\gamma$  is worth nothing if  $\beta$  was computed as  $\gamma$  divided by  $\alpha$ . Compute both sides from the data by separate routes, or do not write the check.

### 6.3 A caution: checks that cannot fail

An assertion that no possible input could violate is documentation, not a test. Ours checked an algebraic identity whose two sides had been computed from each other, so they agreed by construction and the check passed every time it ran. That is class (7) of Section 2, and it is worse than having no check, because the reviewer who sees it reasonably concludes that the identity was verified.

When you add an assertion, ask one question: what input would make this fail? If you cannot name one, delete the line or rewrite it until you can.

## 7. Limitations

Each entry states a boundary and, where the protocol does something about it, what. Where it does not, the entry says so.

**The methods themselves are not re-derived.** The correctness of established packages and published estimators is assumed; re-deriving them is a research exercise rather than quality control. What the census does record, at every point where the analysis meets such a method, is the settings it was called with and the conditions it was applied under, so a method used outside the range it was designed for is visible even when its derivation is not re-examined. An error inside a package is outside the protocol’s reach and is properly the business of that software’s maintainers.

**The fixed inputs are assumed correct.** The census stops at the frozen input file by construction, since that is what bounds the backward walk in Section 4.4. Everything before it is outside the protocol: a data delivery with a column shifted by one, a species code mis-keyed at entry, a gear type recorded under two spellings, a station whose coordinates place it on land. Part A checksums these files, which establishes that every run read the same bytes and nothing whatever about whether those bytes are right. Assuring the data is a separate activity with its own methods, and a pipeline verified by this protocol can compute exactly the intended quantity from a bad input file and report it with confidence.

**Suitability is surfaced, not settled.** Writing down what each node computes, at what grain and in what units, is the material a judgement about suitability requires, and the census forces it into one place where the choices can be compared. Section 4.6 gives two questions raised this way in our own pipeline, neither of which any amount of code review would have produced. But a verdict in the table records that a choice was made and what it was; it does not defend it. The answers are argued in the Methods and Discussion, and a reader is entitled to disagree with a choice the table records as ok.

**Rarely executed code is reached by neither part.** Part A reaches code that produces no quantity when it fires on real data, and the text search reaches it as text, which is why those two sit alongside the census. What nothing here reaches is code guarded by conditions that did not arise in the runs performed and that a search has no distinctive string to match. A pipeline whose behaviour on unusual input matters needs testing of a kind this document does not cover.

**The audit of variation rests on inspection, and its reduced form rests on names.** Part A’s enumeration of the sources of variation is a reading of the code, so it is bounded by what the reader knows about the language and the libraries in use. A draw inside a function not known to draw, or parallelism inside a library that does not advertise it, will pass an audit performed carefully. The scan permitted in Section 3.6 is weaker again, because it finds only mechanisms that name themselves in the text: a random draw inside a helper the analysis wrote itself, a function passed as an argument and called through `do.call`, or a library whose parallel backend is switched on by an option rather than by a call, all leave nothing for a search to match. The same-environment verification run is the partial check on both, since two runs that disagree show that a source was missed. Where only one verification run is affordable, that check is not made.

Delegating the reading pass, as Section 3.3 permits, changes the shape of this limitation without removing it. A model reads more of the code than a hurried author will, but it fails the same way an author does, by omission, and it fails more confidently. It is also least reliable on exactly the question the reading pass exists to answer, since whether a package draws or parallelizes internally is a fact about a specific version that a model may not have. The audit’s protection is therefore the

repeat run rather than the care taken over the reading, delegated or not, and an audit performed without that run rests on inspection alone whoever performed it.

**The agreement rule inherits the uncertainty the manuscript reports.** Section 3.4 judges a difference between runs against the uncertainty reported for that quantity, which makes the rule strict where a number is reported exactly and lenient where an interval is wide. Where the reported interval is itself too wide, because it assumes independence among nested observations or ignores a component of variance, the rule is most permissive exactly where the analysis is weakest. Part A therefore borrows its threshold from something Part B is meant to be checking, and the two parts are less independent than their separation suggests. The practical protection is the instruction in Section 3.4 to treat a difference larger than the audit predicted as a finding whether or not it clears the rule.

**Node boundaries are a judgement, and the anchors pull on them.** Where one node ends and the next begins is decided by whoever enumerates, and a transformation folded into a neighbour is checked less carefully than one standing alone. The granularity rule in Section 4.8 is a convention rather than a definition. A second pressure comes from the anchors themselves: an anchor must be unique in its file, and the easiest way to make one unique is to widen the region, so a region may have been drawn for a findable boundary rather than for a meaningful one, and nothing in the table distinguishes the two. What the table does offer is inspectability. Because each node carries the whole region of code it describes rather than a single line, a reader can see the full extent of what was folded in and judge whether the fold hid a decision.

**Where the enumeration is delegated, the description comes from the same kind of tool as the code.** Pass one of Section 4.4 may be drafted by a language model. The risk is not that the model invents a node but that it describes the code accurately as written and thereby ratifies a misunderstanding: a grouping that omits a variable is described as a grouping by the variables present, and the row reads as correct. A model may also reproduce the analysis's own framing, so a quantity the code calls alpha diversity is recorded as alpha diversity whatever it is computed at. The protocol's answer is the requirement in Section 4.4 that every row be read against the code before any node is checked, but that is a human pass whose thoroughness is not itself recorded, and a description that is wrong in the same way the code is wrong makes an error look confirmed rather than leaving it visible. Where this matters most, the expected values in Section 4.5 must come from outside the code, and that rule is what keeps the delegation from closing the loop entirely.

**The census depends on the analysis being describable, and produces the description where none exists.** Checking that a quantity is the one the Methods name requires Methods specific enough to check against. Where they are vague, there is nothing to compare the code with. In our case the Methods had not been written when the census was built, and the relation ran the other way: the operation, settings and grain\_out fields became the Methods content, and a setting that could not be justified once written down plainly was itself the finding. Either way the check is only as sharp as the description, and a table completed carelessly gives a false sense of coverage.

**The failure taxonomy is empirical and provisional.** Section 2 is drawn from one pipeline and one pair of models, and describes errors that survived interactive development rather than all errors produced. The specification classes in particular are likely incomplete, since they were

assembled from cases we happened to catch. We offer the taxonomy as a search specification to be extended, not as a checklist that is complete when exhausted.

**Effort estimates assume familiarity with the data.** A reviewer who does not know what the numbers should mean cannot perform Part B, because every question at a node is a question about meaning. This makes the protocol something authors do, or that a reviewer does in conversation with them; it is not a task that can be handed to someone outside the project.

## 8. Summary of recommendations

### 8.1 The two questions

- Distinguish implementation, whether the code does what it is described as doing, from specification, whether that description is right. Specification is often harder to check, and is where the errors that change conclusions live.
- Do not treat invented functions and file names as the main risk. Execution excludes them at the first run. What survives is a real function computing a quantity nobody decided on, and only an explicit check on what was computed will find it.

### 8.2 Part A: a trustworthy run

- Require that reported numbers come from a documented run that starts from nothing: no saved workspace, no objects left from earlier work, and a folder holding only the code and the fixed inputs.
- Verify with two further runs, at least one of them on a different machine. Where two are not affordable, perform one, and put it on the different machine.
- Audit the sources of variation before comparing the runs, so that a difference can be read when it appears. The mechanical scan of the code is the part that must not be skipped; the written disposition can be reduced.
- Judge agreement by the uncertainty the manuscript reports: a quantity reported without uncertainty must match exactly, and a quantity reported with it must differ by much less than it. A difference larger than the audit predicted is a finding whether or not it clears the rule.
- Quantify stochastic variation and report it, rather than treating identical output as the goal.

### 8.3 Part B: the node census

- Enumerate every transformation on the path to a reported number rather than selecting a few quantities to verify. Selection is a judgement about what matters, made before the checking that would inform it.
- Enumerate forwards through each script, prune backwards from the reported numbers, and check forwards from the data.
- Record each node's grain and units on both sides, its settings, and its behaviour on missing values. These four are where specification errors show up, and where the analysis's own choices become comparable with one another.

- Give each node the whole region of code it occupies, delimited by lines copied from the source rather than by line numbers, and check mechanically that every anchor still resolves.
- Delegate the enumeration if it helps, but not the checking, and read every delegated row against the code before checking any node.
- Derive the expected value for any hand computation from outside the code being tested. Where a model wrote the analysis, it must not also supply the values the analysis is checked against.

## 8.4 Documentation and reporting

- Iterate until no finding remains uncorrected, and carry the open questions into the Methods and Discussion rather than into a revision of the code.
- Publish the node table and the variation audit, and report what was done in specific, checkable terms rather than in adjectives. Coverage and open questions are the numbers worth reporting; a count of findings is not.
- Where a part was performed in reduced form, say which and why. A reduced form reported plainly is worth more than a full form claimed loosely.

## 8.5 While the code is being written

- Define each shared convention once, in a file every script reads, so that a constant cannot drift between copies.
- Prefer stopping to degrading. An absent input, a lookup that matches nothing, and a category level outside the expected set should halt the run rather than become a default or a missing value. Of every check you add, ask what input would make it fail.

## References

- Anthropic. 2026. Claude [large language model]. Anthropic, San Francisco, California, USA. <https://claude.ai>
- Gronenschild, E. H. B. M., P. Habets, H. I. L. Jacobs, R. Mengelers, N. Rozendaal, J. van Os, and M. Marcelis. 2012. The effects of FreeSurfer version, workstation type, and Macintosh operating system version on anatomical volume and cortical thickness measurements. *PLoS ONE* 7(6): e38234. <https://doi.org/10.1371/journal.pone.0038234>
- Kellner, K. F., J. W. Doser, and J. L. Belant. 2025. Functional R code is rare in species distribution and abundance papers. *Ecology* 106(1): e4475. <https://doi.org/10.1002/ecy.4475>
- OpenAI. 2026. ChatGPT [large language model]. OpenAI, San Francisco, California, USA. <https://chatgpt.com>
- R Core Team. 2026. R: A language and environment for statistical computing. R Foundation for Statistical Computing, Vienna, Austria. <https://www.R-project.org/>
- Trisovic, A., M. K. Lau, T. Pasquier, and M. Crosas. 2022. A large-scale study on research code quality and execution. *Scientific Data* 9: 60. <https://doi.org/10.1038/s41597-022-01143-6>

Wilson, G., D. A. Aruliah, C. T. Brown, N. P. Chue Hong, M. Davis, R. T. Guy, S. H. D. Haddock, K. D. Huff, I. M. Mitchell, M. D. Plumbley, B. Waugh, E. P. White, and P. Wilson. 2014. Best practices for scientific computing. *PLoS Biology* 12(1): e1001745. <https://doi.org/10.1371/journal.pbio.1001745>